

Código:

```
import java.util.Random;

public class GeneradorNumeros {
    public static void main(String[] args) {
        Random random = new Random();

        int sumalImpares = 0;
        int limite = 25;

        // El proceso continúa mientras la suma acumulada sea menor o igual que 25
        while (sumalImpares <= limite) {
            // Genera un número entero al azar entre 0 y 10 (ambos inclusive)
            int numero = random.nextInt(11);

            if (numero % 2 == 0) {
                System.out.println("Número generado: " + numero + " - Es par");
            } else {
                System.out.println("Número generado: " + numero + " - Es impar");
                sumalImpares += numero;
            }
        }

        System.out.println();
        System.out.println("La suma de los números impares es: " + sumalImpares);
        System.out.println("La suma superó el valor " + limite + ". El proceso ha finalizado.");
    }
}
```

Explicación del Código

1. Importación de la librería

Java
import java.util.Random;

Java no incluye la generación de números al azar por defecto dentro del lenguaje base, así que importamos la clase Random del paquete `java.util`. Esta herramienta es la que nos permitirá generar los valores aleatorios.

2. Declaración de la clase y el método principal

Java
public class GeneradorNumeros {

```
public static void main(String[] args) {
```

- `public class GeneradorNumeros`: Define el bloque principal de nuestro programa. Recordá que el archivo en la computadora debe guardarse con este mismo nombre (`GeneradorNumeros.java`).
- `public static void main(String[] args)`: Es el punto de entrada. Todo el código que Java ejecuta al darle "Run" empieza dentro de este método.

3. Creación del generador y las variables

Java

```
Random random = new Random();
```

```
int sumaImpares = 0;
```

```
int limite = 25;
```

- `Random random = new Random()` ;: Instanciamos (creamos) el objeto que va a tirar los números al azar.
- `int sumaImpares = 0` ;: Es nuestro acumulador. Nace en 0 y ahí vamos a ir sumando únicamente los números impares que salgan.
- `int limite = 25` ;: Guardamos la condición de corte en una variable para no usar "números mágicos" sueltos en el código.

4. El bucle de control (while)

Java

```
while (sumaImpares <= limite) {
```

Esta es la estructura clave del programa. Evalúa la condición antes de generar cada número:

- Mientras `sumaImpares` sea menor o igual a 25, el bucle se vuelve a repetir.
- En el instante en que `sumaImpares` pasa a ser 26 o más (después de sumar un impar), el bucle se corta de inmediato y no vuelve a entrar.

5. Generación del número aleatorio

Java

```
int numero = random.nextInt(11);
```

El método `.nextInt(n)` genera enteros al azar desde 0 hasta $n - 1$. Al pasarle 11, nos devuelve un número entero comprendido entre 0 y 10 inclusive (cumpliendo con el rango pedido).

6. Clasificación Par / Impar y acumulación

Java

```
if (numero % 2 == 0) {  
    System.out.println("Número generado: " + numero + " - Es par");  
} else {  
    System.out.println("Número generado: " + numero + " - Es impar");  
    sumaImpares += numero;  
}
```

- `numero % 2 == 0`: Utiliza el operador de residuo/módulo (%). Si al dividir el número entre 2 el resto es 0, el número es par. Solo lo imprimimos en pantalla y no alteramos la suma.
- `else`: Si el resto es distinto de cero, el número es impar. Imprimimos el mensaje correspondiente y ejecutamos `sumaImpares += numero`; (que equivale a `sumaImpares = sumaImpares + numero`), sumándolo al total acumulado.

7. Mensajes de cierre

Java

```
System.out.println();  
System.out.println("La suma de los números impares es: " + sumaImpares);  
System.out.println("La suma superó el valor " + limite + ". El proceso ha finalizado.");
```

Una vez que el bucle `while` se rompe, el flujo del programa llega a estas líneas para mostrar el resultado final acumulado y confirmar que el límite fue superado.